Оглавление

Конспект по pandas · примеры выполнены и проверены на pandas 3.0

Series 3

- Создание Series 3
- Добавление элемента в Series 4
- Удаление элемента из Series 4
- Выборка данных из Series по условию 5
- Обход содержимого Series в цикле 5
- Функции для всего ряда max(), min(), mean() 6
- Арифметические операции с Series 6
- Получение нового ряда из старого 6
- Типы данных в Series 7

Index 7

- Свойства индекса 7
- Переименование и сброс индекса 8
- Выравнивание по индексу и reindex 8

DataFrame 9

- Создание DataFrame 9
- Загрузка данных из файлов 10
- Первичный осмотр данных 11
- Добавить колонку в DataFrame 11
- Переименование колонок в DataFrame 12
- Выборка данных из DataFrame по условию 12
- Выборка определенных колонок и столбцов 12
- Удаление колонки из DataFrame 13
- Табличные вычисления как в Excel 13
- Создание колонки на основе данных в другой колонке 14
- Применение функций к DataFrame 15
- Функции датафрейма вдоль осей 15
- Применение функции вдоль индекса 15
- Перебор таблицы в цикле 16
- Сортировка 16
- Группировка 17
- Установка индекса в DataFrame 18
- Конкатенация датафреймов и индексы 18
- Работа со строками 19
- Дубликаты 20

Работа с пропусками данных 20
Сводные таблицы 21
Соединения таблиц 21
Запись датафреймов в Excel-файл 23
Преобразование даты при загрузке данных из CSV-файла 23
Загрузка и преобразование в одну строку 23

Функции для работы с датами 24
Создание и разбор дат 24
Вычисления на основе даты/времени 24
Фильтрация строк на основе даты 24
Компоненты даты и форматирование 25
Группировка по периодам: resample и Grouper 26
Часовые пояса 26

Строковые методы .str 27
Очистка и преобразование текста 27
Поиск и фильтрация по тексту 27
Разбиение и регулярные выражения 27
Типичная задача: цена в виде текста 28

Сдвиги и скользящие окна 28

Категориальные данные 29

MultiIndex 29

Быстрая визуализация 30
Линия и столбцы 30
Распределение и связь двух величин 31

Особенности pandas 3.0 31

Частые ошибки и подводные камни 32
Присваивание по цепочке и and / or 32
loc и iloc, inplace, NaN 32
Даты и повторяющиеся ключи 33
Скорость и память 33

Шпаргалка 33
Загрузка, осмотр, типы 33
Выборка и фильтрация 34
Преобразование и пропуски 34
Группировка, сводные, соединения 34
Даты, текст, окна 35

Series

Series (далее «ряд») — одномерный набор данных с индексом: что-то среднее между списком и словарём. По ключу (метке индекса) можно получить значение, а над всем рядом сразу выполнять арифметику, фильтрацию и статистику. Каждый столбец DataFrame — это тоже Series. Чтобы создать ряд, задают три параметра:

- **data** — данные ряда (список, словарь, массив numpy, скаляр);
- **index** — индекс, или ключи, по которым потом можно искать данные. Если не задан — числа 0, 1, 2, …;
- **name** — необязательное имя ряда. Становится названием столбца, когда ряды собирают в датафрейм.

Создание Series

Проще всего создать ряд из списка, указав индекс и данные. Индекс — аналог ключей словаря, данные — его значения.

```
import pandas as pd
import numpy as np
from pandas import Series, DataFrame

# create series using lists
gdp_regions = Series(data=[34931, 32095, 23125, 4043, 2722, 2028],
                     index=['Asia', 'North America', 'Europe',
                            'South America', 'Africa', 'Oceania'],
                     name='gdp_regions')

print('GDP for regions:')
print(gdp_regions)
```

```
# вывод
GDP for regions:
Asia             34931
North America    32095
Europe           23125
South America     4043
Africa            2722
Oceania           2028
Name: gdp_regions, dtype: int64
```

Другие способы создания: из словаря (ключи становятся индексом), из скаляра (значение повторяется) и из массива numpy.

```
s_dict = Series({'USA': 28781, 'China': 18532, 'Germany': 4591})   # from dict
s_const = Series(0, index=['a', 'b', 'c'])                         # from scalar
s_np = Series(np.arange(5) * 10)          # default index: RangeIndex 0..4

print(s_dict, s_const, s_np, sep='\n\n')
```

Как получать значения: по метке, по позиции, списком меток и срезом. Обратите внимание на разницу срезов по меткам и по позициям.

```
print(gdp_regions['Asia'])                # by label
print(gdp_regions.iloc[0])                # by position
print(gdp_regions[['Asia', 'Europe']])    # list of labels -> Series
print(gdp_regions['Asia':'Europe'])       # slice by labels: END IS INCLUDED
print(gdp_regions.iloc[:2])               # slice by positions: end excluded
```

Важно. Не используйте `s[0]` для доступа по позиции: поведение зависит от типа индекса и в новых версиях pandas убрано. Для позиции — `iloc`, для меток — `loc` (подробнее в разделе про выборку из DataFrame).

Добавление элемента в Series

Чтобы добавить элемент, нужно указать новый ключ и новое значение. Для добавления сразу нескольких элементов используйте `pd.concat` (метод `Series.append` удалён в pandas 2.0).

```
# add one element: new key and new value
gdp_regions['Antarctica'] = 1
print(gdp_regions.tail(2))

# add several elements at once
extra = Series({'Mars': 0, 'Moon': 0}, name='gdp_regions')
gdp_regions = pd.concat([gdp_regions, extra])
print(gdp_regions.tail(4))
```

Удаление элемента из Series

Воспользуйтесь оператором **del**, чтобы удалить значение по ключу. Метод `drop` возвращает новый ряд и не меняет исходный, а `pop` удаляет ключ и возвращает его значение.

```
# delete data one key/value
del gdp_regions['Mars']
assert 'Mars' not in gdp_regions.keys()

without_moon = gdp_regions.drop('Moon')                  # new Series, old one is intact
without_two = gdp_regions.drop(['Antarctica', 'Moon'])   # several keys

removed = gdp_regions.pop('Moon')       # delete AND return the value
print(removed, len(gdp_regions))

gdp_regions = gdp_regions.drop('Antarctica')    # back to the original six regions
print(len(gdp_regions))
```

Выборка данных из Series по условию

Чтобы выбрать данные из Series, напишите условие в квадратных скобках. Для простых условий можно писать `s[s>1000]`, но можно использовать и отдельную переменную для условия (маску).

```
s = Series(data={
    'USA': 28781,
    'Russia': 2056,
    'China': 18532,
    'South Korea': 1760,
    'France': 3130,
    'Germany': 4591,
    'UK': 3495,
    'Japan': 4110,
    'Canada': 2242
})

# mask/condition: Series of True/False
mask = s > 3000
print(mask.head(3))
print(s[mask])
```

```
# вывод
USA        True
Russia    False
China      True
dtype: bool
USA        28781
China      18532
France      3130
Germany     4591
UK          3495
Japan       4110
dtype: int64
```

Условия комбинируются операторами `&` (и), `|` (или), `~` (не). Каждое условие обязательно берётся в скобки. Слова `and`, `or`, `not` для рядов не работают.

```
print(s[(s > 2000) & (s < 4000)])          # AND
print(s[(s < 2000) | (s > 10000)])         # OR
print(s[~(s > 3000)])                      # NOT
print(s[s.index.isin(['USA', 'Japan'])])   # by list of keys
print(s[s.between(3000, 5000)])            # range, both ends included
print(s[s.index.str.startswith('U')])      # by text of the key
```

Обход содержимого Series в цикле

```
for country, gdp in s.items():        # key and value
    if gdp > 10000:
        print(country, gdp)

total = 0
for gdp in s:                         # values only
    total += gdp
print(total == s.sum())

for pos, country in enumerate(s.index):    # position and key
    if pos == 2:
        print(pos, country)
```

> **Совет.** Цикл по ряду нужен редко: почти всё, что делают в цикле, быстрее и короче выражается векторной операцией (`s * 2`, `s[s > 1000]`, `s.sum()`).

Функции для всего ряда max(), min(), mean()

```
print(s.max(), s.min(), s.mean())
print(s.idxmax(), s.idxmin())              # KEYS of max and min
print(s.sum(), s.median(), round(s.std(), 1))
print(s.nlargest(3))                       # three largest values with keys
print(s.describe())                        # all main statistics at once
```

Арифметические операции с Series

Операции с числом применяются к каждому элементу. При операции между двумя рядами значения выравниваются **по индексу**, а ключи без пары дают NaN. Метод с параметром `fill_value` позволяет заменить отсутствующие значения.

```
print((s / 1000).head(3))                      # number applied to each element

pop = Series({'USA': 335, 'China': 1412, 'Russia': 146})     # million people
ratio = s / pop                                # aligned by index
print(ratio.sort_index())

print(s.add(pop, fill_value=0).sort_index().head(4))
```

Получение нового ряда из старого

```
big = s[s > 3000]                              # filtered copy
in_k = s.apply(lambda x: x / 1000)             # function for every element
size = s.map(lambda x: 'big' if x > 3000 else 'small')   # also accepts a dict
print(size.head(3))

print(s.where(s > 3000, 0).head(4))            # keep if True, else replace
print(s.clip(upper=5000).head(4))              # cap the values
print(s.sort_values(ascending=False).head(3))
print(s.rank(ascending=False).head(4))         # place in the rating
```

Типы данных в Series

Тип ряда хранится в `dtype`. Правильный тип влияет на скорость, память и корректность операций (числа в виде строк не сложить, даты не вычесть).

dtype	Что хранит	Примечание
`int64`, `float64`	целые и вещественные числа	в целых нет NaN: при пропусках ряд станет float64
`bool`	True / False	результат любых условий
`str` (раньше `object`)	текст	в pandas 3 строки имеют отдельный тип `str`
`datetime64`	дата и время	см. раздел про даты
`timedelta64`	разница между датами	результат вычитания дат
`category`	повторяющиеся значения	экономит память, можно задать порядок
`Int64`, `boolean`, `string`	nullable-типы	допускают пропуски `<NA>` без превращения в float

```
print(s.dtype)
s_float = s.astype('float64')

raw = Series(['10', '20', 'n/a', '40'])          # numbers stored as text
num = pd.to_numeric(raw, errors='coerce')        # unparseable -> NaN
print(num)

ints = num.astype('Int64')                       # nullable integer keeps <NA>
print(ints)
print(Series(['a', 'b', 'a']).astype('category').dtype)
```

Index

Index — объект с метками строк (у Series и DataFrame) или столбцов (`df.columns` — тоже Index). Он нужен для быстрого поиска и для выравнивания данных при операциях. Индекс неизменяем: отдельное значение заменить нельзя, но можно создать новый индекс целиком.

Свойства индекса

```
idx = gdp_regions.index
print(idx)
print(idx.name, idx.is_unique, idx.dtype)
print('Asia' in idx, idx.get_loc('Europe'))       # membership and position of a key
print(idx.tolist())

try:
    idx[0] = 'ASIA'                               # index is immutable
except TypeError as e:
    print('Error:', e)
```

Переименование и сброс индекса

```
named = gdp_regions.rename_axis('region')              # give the index a name
named = named.rename(index={'Asia': 'ASIA'})           # rename some keys
print(named.reset_index().head(3))              # index -> column, Series -> DataFrame
print(named.reset_index(drop=True).head(3))            # just throw the index away
```

Выравнивание по индексу и reindex

`reindex` приводит ряд к заданному набору ключей: недостающие получают NaN (или значение `fill_value`), лишние отбрасываются. Индекс может содержать повторы — тогда выборка по такому ключу возвращает несколько значений.

```
order = ['Europe', 'Asia', 'Antarctica']
print(gdp_regions.reindex(order))
print(gdp_regions.reindex(order, fill_value=0))
print(gdp_regions.sort_index().head(3))

dup = Series([1, 2, 3], index=['a', 'a', 'b'])
print(dup['a'])                # two values -> Series
print(dup['b'], dup.index.is_unique)
```

DataFrame

DataFrame — двумерная таблица: строки с индексом, столбцы с названиями, у каждого столбца свой тип. Это набор рядов (Series) с общим индексом. В примерах дальше используется датафрейм `df` с заказами интернет-магазина: он создаётся в первом разделе.

Создание DataFrame

Самый частый способ — словарь «название столбца: список значений».

```
# create dataframe using dict of lists
df = DataFrame({
    'order_id': [101, 102, 103, 104, 105, 106, 107, 108],
    'date': ['2026-01-05', '2026-01-17', '2026-02-03', '2026-02-21',
             '2026-03-09', '2026-03-14', '2026-04-02', '2026-04-25'],
    'region': ['Moscow', 'Kazan', 'Moscow', 'Perm',
               'Kazan', 'Moscow', 'Perm', 'Kazan'],
    'product': ['laptop', 'phone', 'phone', 'tablet',
                'laptop', 'tablet', 'phone', 'laptop'],
    'qty': [2, 5, 3, 4, 1, 2, 6, 3],
    'price': [950, 400, 420, 300, 980, 310, 390, 940],
})
print(df)
```

```
# вывод
   order_id        date  region product  qty  price
0       101  2026-01-05  Moscow  laptop    2    950
1       102  2026-01-17   Kazan   phone    5    400
2       103  2026-02-03  Moscow   phone    3    420
3       104  2026-02-21    Perm  tablet    4    300
4       105  2026-03-09   Kazan  laptop    1    980
5       106  2026-03-14  Moscow  tablet    2    310
6       107  2026-04-02    Perm   phone    6    390
7       108  2026-04-25   Kazan  laptop    3    940
```

Другие способы: список словарей (типично для JSON и API), список списков, набор рядов и массив numpy.

```
rows = [{'name': 'Anna', 'age': 31}, {'name': 'Oleg', 'age': 27}]    # one dict = one row
print(DataFrame(rows))

print(DataFrame([['Anna', 31], ['Oleg', 27]], columns=['name', 'age']))

gdp = Series([34931, 32095], index=['Asia', 'Europe'], name='gdp')
pop = Series([4700, 750], index=['Asia', 'Europe'], name='pop')
print(pd.concat([gdp, pop], axis=1))                  # series -> columns

print(DataFrame(np.arange(6).reshape(2, 3), columns=['a', 'b', 'c'], index=['x', 'y']))
```

Загрузка данных из файлов

На практике датафрейм почти всегда создают чтением файла. Для CSV — `pd.read_csv`, для Excel — `pd.read_excel`. Главные параметры, которые приходится указывать:

Параметр	Назначение	Пример
`sep`	разделитель столбцов	`sep=";"` (русский Excel сохраняет с `;`)
`decimal`, `thousands`	десятичный разделитель и разделитель тысяч	`decimal=","`
`encoding`	кодировка файла	`"utf-8"`, `"utf-8-sig"`, `"cp1251"`
`usecols`	читать только нужные столбцы	`usecols=["date", "amount"]`
`dtype`	явно задать типы	`dtype={"id": "str"}`
`parse_dates`	превратить столбцы в даты	`parse_dates=["date"]`
`na_values`	что считать пропуском	`na_values=["n/a", "-"]`
`header`, `names`	строка заголовка / свои названия	`header=None, names=[...]`
`skiprows`, `nrows`	пропустить строки / прочитать N	`nrows=1000`
`index_col`	столбец, который станет индексом	`index_col="order_id"`
`chunksize`	читать файл частями	`chunksize=100_000`

```
import io

# in real life: pd.read_csv('data.csv', ...) -- here the "file" is a string
csv_text = """order_id;date;region;qty;price
101;05.01.2026;Moscow;2;950,5
102;17.01.2026;Kazan;5;400
103;03.02.2026;Moscow;3;n/a
"""
raw = pd.read_csv(io.StringIO(csv_text), sep=';', decimal=',', na_values=['n/a'])
print(raw)
print(raw.dtypes)
```

```
# вывод
   order_id        date  region  qty  price
0       101  05.01.2026  Moscow    2  950.5
1       102  17.01.2026   Kazan    5  400.0
2       103  03.02.2026  Moscow    3    NaN
order_id      int64
date            str
region          str
qty           int64
price       float64
dtype: object
```

Файлы Excel читаются так же; можно выбрать лист и столбцы. Для нескольких листов сразу передайте `sheet_name=None` — получите словарь датафреймов.

```
print(pd.read_csv('sales.csv', usecols=['order_id', 'region', 'qty'], nrows=3))
print(pd.read_excel('sales.xlsx', sheet_name='orders', usecols='A:C', nrows=3))
print(list(pd.read_excel('sales.xlsx', sheet_name=None).keys()))     # all sheets
```

Справка. Другие форматы читаются аналогично: `pd.read_json`, `pd.read_sql(query, connection)`, `pd.read_parquet` (нужен pyarrow), `pd.read_html`, `pd.read_clipboard`. Для больших файлов parquet быстрее и компактнее CSV.

Первичный осмотр данных

Первое, что делают после загрузки, — смотрят на форму таблицы, типы, пропуски и разброс значений.

```
print(df.head(3))                    # first rows (5 by default)
print(df.tail(2))                    # last rows
print(df.sample(2, random_state=0))  # random rows
print(df.shape)                      # (rows, columns)
df.info()                            # types, non-null counts, memory
```

```
print(df.describe())                        # statistics of numeric columns
print(df[['region', 'product']].describe())  # text: count, unique, top, freq
print(df['region'].value_counts())          # how often each value appears
print(df['region'].nunique(), df['product'].unique())
print(df.isna().sum())                      # missing values per column
```

```
# вывод
        order_id       qty       price
count    8.00000  8.000000    8.000000
mean   104.50000  3.250000  586.250000
std      2.44949  1.669046  309.743558
min    101.00000  1.000000  300.000000
25%    102.75000  2.000000  370.000000
50%    104.50000  3.000000  410.000000
75%    106.25000  4.250000  942.500000
max    108.00000  6.000000  980.000000
        region product
count        8       8
unique       3       3
top     Moscow  laptop
freq         3       3
region
Moscow    3
Kazan     3
Perm      2
Name: count, dtype: int64
3 <StringArray>
…
```

Добавить колонку в DataFrame

```
df['revenue'] = df['qty'] * df['price']         # from other columns
df['currency'] = 'USD'                          # constant for all rows
df.insert(2, 'year', 2026)                      # at a chosen position (0-based)
with_vat = df.assign(vat=lambda d: d['revenue'] * 0.2)   # new DataFrame, chain-friendly
print(with_vat.head(3))
```

Важно. Новая колонка из списка или массива должна иметь ту же длину, что и таблица. Ряд (Series) подставляется **по индексу**, а не по порядку.

Переименование колонок в DataFrame

```
df = base_df.copy()
df = df.rename(columns={'qty': 'quantity', 'price': 'unit_price'})   # selected columns
print(df.columns.tolist())

df.columns = ['id', 'date', 'region', 'product', 'qty', 'price', 'revenue']  # all at once
df = df.rename(columns=str.upper)                                    # by function
print(df.columns.tolist())
print(df.add_prefix('x_').columns.tolist()[:3])

# typical cleaning of messy headers coming from files
messy = DataFrame(columns=['Order ID ', 'Total Price', 'Region Name'])
messy.columns = messy.columns.str.strip().str.lower().str.replace(' ', '_')
print(messy.columns.tolist())
```

Выборка данных из DataFrame по условию

Условие даёт булеву маску (True/False для каждой строки), которой фильтруют таблицу. Правила те же, что у Series: `&`, `|`, `~` и скобки вокруг каждого условия.

```
print(df[df['region'] == 'Moscow'])
print(df[(df['qty'] >= 3) & (df['price'] > 400)])
print(df[(df['region'] == 'Perm') | (df['product'] == 'tablet')])
```

```
print(df[df['region'].isin(['Kazan', 'Perm'])]['order_id'].tolist())
print(df[df['price'].between(300, 400)]['order_id'].tolist())      # both ends included
print(df[~df['product'].isin(['phone'])]['order_id'].tolist())     # NOT in list
print(df[df['product'].str.contains('top')]['order_id'].tolist())  # by text

# the same with query(): readable for long conditions
print(df.query('qty >= 3 and region == "Kazan"')['order_id'].tolist())
min_qty = 4
print(df.query('qty >= @min_qty')['order_id'].tolist())            # @ = Python variable
```

Выборка определенных колонок и столбцов

Одна колонка в одинарных скобках даёт Series, список колонок в двойных скобках — DataFrame. Для выбора строк и столбцов вместе используют `loc` (по меткам) и `iloc` (по позициям).

	`loc`	`iloc`
Адресация	по меткам индекса и названиям колонок	по номерам строк и колонок (с нуля)
Срез `a:b`	конец **включается**	конец **не включается**
Булева маска	можно	только массив, не Series
Пример	`df.loc[1:3, "qty"]`	`df.iloc[1:3, 4]`

```
print(type(df['qty']).__name__, type(df[['qty']]).__name__)    # Series vs DataFrame
print(df[['region', 'revenue']].head(3))                        # several columns
print(df.loc[2, 'region'])                          # row label 2, column 'region'
print(df.loc[1:3, ['region', 'qty']])               # END INCLUDED: rows 1, 2, 3
print(df.iloc[1:3, [2, 4]])                         # end excluded: rows 1, 2
print(df.loc[df['qty'] > 3, ['order_id', 'qty']])  # mask + columns
print(df.iloc[-2:])                                 # last two rows
print(df.at[0, 'price'], df.iat[0, 5])              # fast access to a single cell
print(df.filter(like='r').columns.tolist())         # columns by part of the name
print(df.select_dtypes(include='number').columns.tolist())
```

Удаление колонки из DataFrame

```
df = base_df.copy()
df = df.drop(columns=['revenue'])                # recommended: returns a new table
df = df.drop(columns=['product', 'qty'])         # several columns
del df['price']                                  # in place
col = df.pop('region')                           # delete AND get the column
df = df.drop(index=[0, 1])                       # drop ROWS by index labels
df = df.drop(columns=['nonexistent'], errors='ignore')    # no error if absent
print(df.head(3))
print(base_df.dropna(axis=1, how='all').shape)    # drop columns that are fully empty
```

Табличные вычисления как в Excel

Формулы Excel, которые «протягивают» вниз, в pandas записываются одной операцией над целым столбцом.

```
df['revenue'] = df['qty'] * df['price']                # =B2*C2, dragged down
df['share'] = (df['revenue'] / df['revenue'].sum()).round(3)    # share of the total
df['cum_revenue'] = df['revenue'].cumsum()             # running total

print(df.loc[df['region'] == 'Moscow', 'revenue'].sum())       # SUMIF
print((df['qty'] > 2).sum())                                    # COUNTIF
print(df.loc[df['product'] == 'phone', 'price'].mean())        # AVERAGEIF
df['size'] = np.where(df['revenue'] > 1000, 'big', 'small')     # IF
df['category'] = df['product'].map({'laptop': 'computers',      # VLOOKUP
                                    'phone': 'mobile', 'tablet': 'mobile'})
print(df[['order_id', 'revenue', 'share', 'cum_revenue', 'size', 'category']])
```

Excel	pandas
СУММ, СРЗНАЧ, МАКС	`df["a"].sum()`, `.mean()`, `.max()`
СУММЕСЛИ / СЧЁТЕСЛИ	`df.loc[условие, "a"].sum()`, `(условие).sum()`
ЕСЛИ	`np.where(условие, a, b)`, `np.select`
ВПР (VLOOKUP)	`df["a"].map(словарь)` или `df.merge(...)`
Сводная таблица	`df.pivot_table(...)`
Фильтр	`df[условие]`, `df.query("...")`
Сортировка	`df.sort_values("a")`
Удалить дубликаты	`df.drop_duplicates()`
Текст по столбцам	`df["a"].str.split(" ", expand=True)`
Объединить текст (&)	`df["a"] + " " + df["b"]`

Создание колонки на основе данных в другой колонке

```
df = base_df.copy()
df['category'] = df['product'].map({'laptop': 'computers',
                                    'phone': 'mobile', 'tablet': 'mobile'})
df['big'] = np.where(df['revenue'] >= 1000, 'yes', 'no')              # two outcomes

conditions = [df['revenue'] < 1000, df['revenue'] < 2000]           # several outcomes
df['level'] = np.select(conditions, ['low', 'mid'], default='high')

df['price_band'] = pd.cut(df['price'], bins=[0, 400, 800, 2000],     # bins by edges
                          labels=['cheap', 'normal', 'premium'])
df['qty_group'] = pd.qcut(df['qty'], q=2, labels=['few', 'many'])    # bins by quantiles
df['code'] = df['region'].str[:3].str.upper()                        # from text
cols = ['product', 'revenue', 'category', 'big', 'level', 'price_band', 'qty_group', 'code']
print(df[cols])
```

```
# вывод
  product  revenue   category  big level price_band qty_group code
0  laptop     1900  computers  yes   mid    premium       few  MOS
1   phone     2000     mobile  yes  high      cheap      many  KAZ
2   phone     1260     mobile  yes   mid     normal       few  MOS
3  tablet     1200     mobile  yes   mid      cheap      many  PER
4  laptop      980  computers   no   low    premium       few  KAZ
5  tablet      620     mobile   no   low      cheap       few  MOS
6   phone     2340     mobile  yes  high      cheap      many  PER
7  laptop     2820  computers  yes  high    premium       few  KAZ
```

Применение функций к DataFrame

`apply` применяет функцию к каждому столбцу (`axis=0`) или к каждой строке (`axis=1`); `map` — к каждому отдельному значению таблицы (раньше назывался `applymap`); `agg` считает несколько статистик; `pipe` встраивает свою функцию в цепочку.

```
df = base_df.copy()
num = df[['qty', 'price', 'revenue']]

print(num.apply(lambda col: col.max() - col.min()))     # per column
print(num.apply(sum, axis=1).head(3))                   # per row
df['label'] = df.apply(lambda r: f"{r['region']}-{r['product']}", axis=1)   # row by row
print(df['label'].head(3).tolist())
print(num.map(lambda x: x * 2).head(2))                 # every value (was applymap)
print(num.agg(['min', 'max', 'mean']))                  # several statistics

def add_k(data, col):                                   # own function inside a chain
    return data.assign(**{col + '_k': data[col] / 1000})
print(df.pipe(add_k, 'revenue')[['revenue', 'revenue_k']].head(3))
```

> **Производительность.** `apply(axis=1)` вызывает Python-функцию для каждой строки и работает медленно. Если задачу можно записать столбцовыми операциями (`df["a"] * df["b"]`, `np.where`, `.str`, `.dt`) — делайте так.

Функции датафрейма вдоль осей

Многие методы принимают параметр **axis**: `axis=0` (по умолчанию) — вдоль строк, результат по одному значению на каждый **столбец**; `axis=1` — вдоль столбцов, результат по одному значению на каждую **строку**.

```
          qty   price   revenue
  0         2     950      1900   → axis=1: across, one result for each ROW
  1         5     400      2000   → axis=1
  ↓
axis=0: down the rows, one result for each COLUMN
```

```
print(num.sum())                    # axis=0: one value per column
print(num.sum(axis=1).head(3))      # axis=1: one value per row
print(num.mean(axis='columns').head(3).round(1))   # the same as axis=1
print(num.idxmax())                 # index label of the maximum in each column
```

Применение функции вдоль индекса

```
d = base_df.set_index('order_id')
print(d.index.map(lambda i: i - 100)[:4])               # function over index values
d.index = d.index.map(lambda i: f'ORD-{i}')             # new labels
print(d.head(2))
print(d.rename(index=str.lower).head(2).index.tolist())   # rename via function
print(d.groupby(lambda label: int(label[-1]) % 2)['qty'].sum())   # group by index function
print(d[d.index.map(lambda i: i.endswith(('1', '3')))].index.tolist())  # filter by index
```

Перебор таблицы в цикле

```
df = base_df.copy()
for idx, row in df.head(2).iterrows():          # slow; row is a Series (types get mixed)
    print(idx, row['region'], row['revenue'])

for row in df.head(2).itertuples():             # faster; row is a namedtuple
    print(row.Index, row.region, row.revenue)

for region, qty in zip(df['region'].head(2), df['qty'].head(2)):   # fastest simple way
    print(region, qty)

for name, column in df[['qty', 'price']].items():     # by columns
    print(name, column.sum())

for region, part in df.groupby('region'):             # by groups
    print(region, len(part))
```

Сортировка

```
print(df.sort_values('revenue', ascending=False).head(3)[['order_id', 'revenue']])
print(df.sort_values(['region', 'revenue'], ascending=[True, False])[['region', 'revenue']])
print(df.sort_index(ascending=False).head(2)[['order_id']])
print(df.nlargest(3, 'revenue')[['order_id', 'revenue']])     # faster than sort + head
print(df.sort_values('product', key=lambda s: s.str.len()).head(3)['product'].tolist())
df['rank'] = df['revenue'].rank(ascending=False, method='dense')   # place in rating
print(df['rank'].tolist())
```

Пропуски при сортировке по умолчанию уходят в конец; `na_position="first"` ставит их в начало. `sort_values` возвращает новую таблицу, поэтому результат нужно сохранить.

Группировка

`groupby` делит таблицу на группы по значениям ключа, считает показатели внутри каждой группы и собирает результат (split — apply — combine).

```
print(df.groupby('region')['revenue'].sum())
print(df.groupby('region')['revenue'].agg(['sum', 'mean', 'count']))

# named aggregation: new column name = (source column, function)
res = df.groupby('region').agg(
    orders=('order_id', 'count'),
    total_qty=('qty', 'sum'),
    avg_price=('price', 'mean'),
    max_revenue=('revenue', 'max'),
)
print(res)
```

```
# вывод
region
Kazan     5800
Moscow    3780
Perm      3540
Name: revenue, dtype: int64
         sum         mean  count
region
Kazan   5800  1933.333333      3
Moscow  3780  1260.000000      3
Perm    3540  1770.000000      2
        orders  total_qty   avg_price  max_revenue
region
Kazan        3          9  773.333333         2820
Moscow       3          7  560.000000         1900
Perm         2         10  345.000000         2340
```

```
print(df.groupby(['region', 'product'])['revenue'].sum())       # two keys
print(df.groupby('region', as_index=False)['revenue'].sum())    # key stays a column

# transform returns a column of the ORIGINAL length: handy for shares in groups
region_total = df.groupby('region')['revenue'].transform('sum')
df['region_share'] = (df['revenue'] / region_total).round(2)
print(df[['region', 'revenue', 'region_share']].head(4))

big = df.groupby('region').filter(lambda g: g['revenue'].sum() > 4000)   # filter groups
print(big['region'].unique())
print(df.groupby('region').size())          # rows per group (count() skips NaN)
```

Установка индекса в DataFrame

`set_index` делает столбец (или несколько) индексом, `reset_index` возвращает индекс обратно в столбец. Индекс по уникальному ключу удобен для поиска строки через `loc`.

```
d = base_df.set_index('order_id')
print(d.head(2))
print(d.loc[103, ['region', 'revenue']])         # row by its key
d = d.reset_index()                              # index -> column again

multi = base_df.set_index(['region', 'product']).sort_index()    # two-level index
print(multi.loc['Kazan', ['qty']])
print(base_df.set_index('date', drop=False).head(2).index.tolist())   # keep the column too

print(base_df.set_index('region').index.is_unique)      # check that the keys are unique
```

Конкатенация датафреймов и индексы

`pd.concat` склеивает таблицы: по умолчанию друг под другом (`axis=0`), с `axis=1` — рядом, выравнивая строки **по индексу**.

```
a = base_df.iloc[:2][['order_id', 'region', 'qty']]
b = base_df.iloc[2:4][['order_id', 'region', 'qty']]

print(pd.concat([a, b]))                          # indexes are kept as they were
print(pd.concat([a, b], ignore_index=True))       # new index 0..n-1
print(pd.concat([a, b], keys=['first', 'second'])) # label of the source -> MultiIndex
```

```
# вывод
   order_id  region  qty
0       101  Moscow    2
1       102   Kazan    5
2       103  Moscow    3
3       104    Perm    4
   order_id  region  qty
0       101  Moscow    2
1       102   Kazan    5
2       103  Moscow    3
3       104    Perm    4
          order_id  region  qty
first  0       101  Moscow    2
       1       102   Kazan    5
second 2       103  Moscow    3
       3       104    Perm    4
```

```
left = DataFrame({'qty': [1, 2, 3]}, index=['a', 'b', 'c'])
right = DataFrame({'price': [10, 20, 30]}, index=['b', 'c', 'd'])
print(pd.concat([left, right], axis=1))                 # all keys, NaN where no pair
print(pd.concat([left, right], axis=1, join='inner'))   # only common keys

print(pd.concat([a, b[['order_id', 'qty']]]))           # different columns -> NaN
```

```
# вывод
   qty  price
a  1.0    NaN
b  2.0   10.0
c  3.0   20.0
d  NaN   30.0
   qty  price
b    2     10
c    3     20
   order_id  region  qty
0       101  Moscow    2
1       102   Kazan    5
2       103     NaN    3
3       104     NaN    4
```

Работа со строками

```
df = base_df.copy()
df.loc[len(df)] = [109, '2026-05-01', 'Perm', 'phone', 1, 410, 410]   # add by new label

new = DataFrame([{'order_id': 110, 'date': '2026-05-03', 'region': 'Kazan',
                  'product': 'tablet', 'qty': 2, 'price': 290, 'revenue': 580}])
df = pd.concat([df, new], ignore_index=True)           # better for several rows
print(df.tail(3))

df = df.drop(index=[0, 1])                             # delete rows by label
df = df[df['qty'] > 1]                                 # delete = keep the others
df = df.reset_index(drop=True)
print(df.shape)
print(df.sample(frac=0.3, random_state=1)['order_id'].tolist())    # random 30%
print(df.iloc[[0, -1]]['order_id'].tolist())                       # first and last
```

Дубликаты

```
dup = DataFrame({'client': ['Anna', 'Oleg', 'Anna', 'Anna'],
                 'city': ['Perm', 'Kazan', 'Perm', 'Omsk'],
                 'sum': [10, 20, 10, 15]})
print(dup.duplicated().tolist())                      # True = repeats a previous row
print(dup.duplicated(subset=['client'], keep=False).tolist())   # mark ALL repeats
print(dup.drop_duplicates())                          # keep the first (default)
print(dup.drop_duplicates(subset='client', keep='last'))
print(dup['client'].is_unique, dup['client'].nunique())
```

```
# вывод
[False, False, True, False]
[True, False, True, True]
  client   city  sum
0   Anna   Perm   10
1   Oleg  Kazan   20
3   Anna   Omsk   15
  client   city  sum
1   Oleg  Kazan   20
3   Anna   Omsk   15
False 2
```

Работа с пропусками данных

Пропуски в pandas обозначаются `NaN` (а в nullable-типах и датах — `<NA>`, `NaT`). Сравнивать их через `==` нельзя: для проверки нужны `isna()` и `notna()`.

```
gaps = DataFrame({'name': ['Anna', 'Oleg', None, 'Ivan'],
                  'age': [31, np.nan, 25, np.nan],
                  'city': ['Perm', 'Kazan', None, 'Omsk']})
print(gaps.isna().sum())                  # how many missing in each column
print(gaps.isna().mean().round(2))        # share of missing values
print(gaps.dropna())                      # drop rows with ANY missing value
print(gaps.dropna(subset=['age']))        # only if 'age' is missing
print(gaps.dropna(thresh=2))              # keep rows having at least 2 values
```

```
# вывод
name    1
age     2
city    1
dtype: int64
name    0.25
age     0.50
city    0.25
dtype: float64
   name   age  city
0  Anna  31.0  Perm
   name   age  city
0  Anna  31.0  Perm
2   NaN  25.0   NaN
   name   age   city
0  Anna  31.0   Perm
1  Oleg   NaN  Kazan
3  Ivan   NaN   Omsk
```

```
print(gaps.fillna({'age': gaps['age'].mean(), 'city': 'unknown'}))   # per column
print(gaps['age'].ffill().tolist())               # carry the previous value forward
print(gaps['age'].interpolate().tolist())         # linear interpolation
print(gaps['city'].replace('Omsk', 'Tomsk').tolist())
print(gaps['name'].isna().tolist(), (gaps['age'] == np.nan).sum())   # == never finds NaN
```

Сводные таблицы

```
print(pd.pivot_table(df, index='region', columns='product', values='revenue',
                     aggfunc='sum', fill_value=0, margins=True))   # margins = totals

print(df.pivot_table(index='region', values=['qty', 'revenue'],
                     aggfunc={'qty': 'sum', 'revenue': ['mean', 'max']}))
```

```
# вывод
product  laptop  phone  tablet    All
region
Kazan      3800   2000       0   5800
Moscow     1900   1260     620   3780
Perm          0   2340    1200   3540
All        5700   5600    1820  13120
       qty revenue
       sum     max         mean
region
Kazan    9    2820  1933.333333
Moscow   7    1900  1260.000000
Perm    10    2340  1770.000000
```

```
print(pd.crosstab(df['region'], df['product']))                       # counts
print(pd.crosstab(df['region'], df['product'], normalize='index').round(2))  # row shares

# melt: wide -> long;  pivot: long -> wide
wide = DataFrame({'region': ['A', 'B'], 'q1': [10, 20], 'q2': [30, 40]})
long = wide.melt(id_vars='region', var_name='quarter', value_name='sales')
print(long)
print(long.pivot(index='region', columns='quarter', values='sales'))

res = df.groupby(['region', 'product'])['qty'].sum()
print(res.unstack(fill_value=0))           # inner index level -> columns
```

Соединения таблиц

`merge` объединяет таблицы по значениям ключевых столбцов (как JOIN в SQL). Тип соединения задаёт параметр `how`.

how	Какие строки остаются	SQL
`inner` (по умолчанию)	только ключи, которые есть в обеих таблицах	INNER JOIN
`left`	все строки левой, из правой — совпавшие	LEFT JOIN
`right`	все строки правой, из левой — совпавшие	RIGHT JOIN
`outer`	все строки из обеих таблиц	FULL OUTER JOIN
`cross`	все пары строк (декартово произведение)	CROSS JOIN

```
orders = DataFrame({'order_id': [1, 2, 3, 4], 'client_id': [10, 11, 10, 99],
                    'amount': [100, 250, 80, 40]})
clients = DataFrame({'client_id': [10, 11, 12], 'name': ['Anna', 'Oleg', 'Ivan']})

print(orders.merge(clients, on='client_id'))                    # inner
print(orders.merge(clients, on='client_id', how='left'))        # keep all orders
print(orders.merge(clients, on='client_id', how='outer', indicator=True))
```

```
# вывод
   order_id  client_id  amount  name
0         1         10     100  Anna
1         2         11     250  Oleg
2         3         10      80  Anna
   order_id  client_id  amount  name
0         1         10     100  Anna
1         2         11     250  Oleg
2         3         10      80  Anna
3         4         99      40   NaN
   order_id  client_id  amount  name      _merge
0       1.0         10   100.0  Anna        both
1       3.0         10    80.0  Anna        both
2       2.0         11   250.0  Oleg        both
3       NaN         12     NaN  Ivan  right_only
4       4.0         99    40.0   NaN   left_only
```

```
clients2 = clients.rename(columns={'client_id': 'id'})          # keys named differently
print(orders.merge(clients2, left_on='client_id', right_on='id').drop(columns='id'))

print(orders.join(clients.set_index('client_id'), on='client_id'))   # join: right by index

try:                                                             # protect from duplicates
    orders.merge(clients, on='client_id', validate='one_to_one')
except Exception as e:
    print(type(e).__name__, '-', e)
```

```
# вывод
   order_id  client_id  amount  name
0         1         10     100  Anna
1         2         11     250  Oleg
2         3         10      80  Anna
   order_id  client_id  amount  name
0         1         10     100  Anna
1         2         11     250  Oleg
2         3         10      80  Anna
3         4         99      40   NaN
MergeError - Merge keys are not unique in left dataset; not a one-to-one merge
Duplicates in left:
  client_id
         10 ...
```

Важно. Если ключ в обеих таблицах повторяется, `merge` вернёт все комбинации, и число строк неожиданно вырастет. Проверяйте `len()` до и после и используйте `validate="one_to_one"`, `"one_to_many"`, `"many_to_one"`.

Запись датафреймов в Excel-файл

```
summary = df.groupby('region', as_index=False)['revenue'].sum()
summary.to_excel('report.xlsx', sheet_name='Summary', index=False)   # one sheet

with pd.ExcelWriter('report_full.xlsx') as writer:                    # several sheets
    df.to_excel(writer, sheet_name='Data', index=False)
    summary.to_excel(writer, sheet_name='Summary', index=False)
    pd.pivot_table(df, index='region', columns='product',
                   values='qty', aggfunc='sum').to_excel(writer, sheet_name='Pivot')

with pd.ExcelFile('report_full.xlsx') as xf:
    print(xf.sheet_names)

# CSV for Russian Excel: separator ';', comma decimal, BOM for correct encoding
df.to_csv('report.csv', index=False, sep=';', decimal=',', encoding='utf-8-sig')
```

Без `index=False` индекс запишется отдельным столбцом. Для форматирования (ширина колонок, заливка) используйте движок `openpyxl` или `xlsxwriter` через `writer.book` и `writer.sheets`.

Преобразование даты при загрузке данных из CSV-файла

Если не указать, что столбец содержит даты, он загрузится как текст. `parse_dates` превращает его в `datetime64`, а `date_format` задаёт формат — это важно для дат вида 05.01.2026, где день и месяц легко перепутать.

```
csv_dates = """date,amount
05.01.2026,100
17.01.2026,250
03.02.2026,80
"""
d = pd.read_csv(io.StringIO(csv_dates), parse_dates=['date'], date_format='%d.%m.%Y')
print(d.dtypes)
print(d['date'].dt.month.tolist())

d2 = pd.read_csv(io.StringIO(csv_dates))                # dates stay text
d2['date'] = pd.to_datetime(d2['date'], format='%d.%m.%Y')   # convert after loading
d3 = pd.read_csv(io.StringIO(csv_dates), parse_dates=['date'], dayfirst=True,
                 index_col='date')                      # shortcut: day first, as index
print(d3.index.min())
```

Загрузка и преобразование в одну строку

Цепочка методов позволяет прочитать данные и сразу привести их в порядок, не создавая промежуточных переменных. Каждый метод возвращает новый датафрейм, поэтому шаги идут один за другим.

```
result = (
    pd.read_csv(io.StringIO(csv_dates), parse_dates=['date'], date_format='%d.%m.%Y')
      .rename(columns=str.upper)
      .assign(AMOUNT_K=lambda t: t['AMOUNT'] / 1000)
      .query('AMOUNT > 90')
      .sort_values('DATE', ascending=False)
      .reset_index(drop=True)
)
print(result)
```

Функции для работы с датами

Создание и разбор дат

`pd.to_datetime` превращает текст в даты. Параметр `errors="coerce"` заменяет нераспознанные значения на `NaT` (аналог NaN для дат) вместо ошибки.

```
df['date'] = pd.to_datetime(df['date'])                    # text -> datetime64
print(df['date'].dtype)
print(pd.to_datetime('05.01.2026', format='%d.%m.%Y'))     # explicit format
print(pd.to_datetime(['2026-01-05', 'oops'], errors='coerce'))
print(pd.Timestamp('2026-03-15 14:30'))                    # a single moment
print(pd.date_range('2026-01-01', periods=4, freq='D'))    # range of dates
print(pd.date_range('2026-01-31', '2026-05-31', freq='ME'))   # month ends
```

Вычисления на основе даты/времени

Разница двух дат — `Timedelta`. Сдвиги задают через `Timedelta` (точные интервалы: дни, часы) или `DateOffset` / `pd.offsets` (календарные: месяцы, конец месяца, рабочие дни).

```
d = pd.Timestamp('2026-03-15')
print(d + pd.Timedelta(days=10))
print(d + pd.DateOffset(months=1))
print(d - pd.Timestamp('2026-01-01'), (d - pd.Timestamp('2026-01-01')).days)

df['days_ago'] = (pd.Timestamp('2026-05-01') - df['date']).dt.days
df['next_week'] = df['date'] + pd.Timedelta(weeks=1)
df['month_end'] = df['date'] + pd.offsets.MonthEnd(0)             # roll to month end
df['month_start'] = df['date'].dt.to_period('M').dt.start_time    # first day of month
df['plus_3_bd'] = df['date'] + pd.offsets.BDay(3)                 # 3 business days
print(df[['date', 'days_ago', 'next_week', 'month_end',
          'month_start', 'plus_3_bd']].head(3))
```

```
# вывод
2026-03-25 00:00:00
2026-04-15 00:00:00
73 days 00:00:00 73
        date  days_ago  next_week  month_end month_start  plus_3_bd
0 2026-01-05       116 2026-01-12 2026-01-31  2026-01-01 2026-01-08
1 2026-01-17       104 2026-01-24 2026-01-31  2026-01-01 2026-01-21
2 2026-02-03        87 2026-02-10 2026-02-28  2026-02-01 2026-02-06
```

Фильтрация строк на основе даты

```
cols = ['order_id', 'date']
print(df[df['date'] >= '2026-03-01'][cols])                   # compare with a string
print(df[df['date'].between('2026-02-01', '2026-02-28')][cols])
print(df[df['date'].dt.month == 3][cols])                     # by component
print(df[df['date'].dt.dayofweek >= 5][cols])                 # weekends (Mon = 0)
```

Если столбец с датой сделать индексом и отсортировать, откроются срезы по частям даты: целый месяц, год или диапазон (оба конца включаются).

```
ts = df.set_index('date').sort_index()
print(ts.loc['2026-02', ['order_id', 'revenue']])                # the whole month
print(ts.loc['2026-01-15':'2026-03-01', ['order_id']].index.tolist()) # date slice
```

Компоненты даты и форматирование

```
df['year'] = df['date'].dt.year
df['month'] = df['date'].dt.month
df['weekday'] = df['date'].dt.day_name()
df['quarter'] = df['date'].dt.quarter
df['week'] = df['date'].dt.isocalendar().week
df['month_last'] = df['date'].dt.is_month_end
print(df[['date', 'year', 'month', 'weekday', 'quarter', 'week']].head(3))
print(df['date'].dt.strftime('%d.%m.%Y').head(2).tolist())          # date -> text
print(df['date'].dt.to_period('M').head(2).tolist())                # month period
```

Код	Значение	Код	Значение
%Y	год, 4 цифры (2026)	%H	часы 00–23
%y	год, 2 цифры (26)	%M	минуты
%m	месяц 01–12	%S	секунды
%d	день 01–31	%A	день недели (название)
%B	название месяца	%j	номер дня в году

Справка. Названия дней недели и месяцев по умолчанию английские. Для русских: `.dt.day_name(locale="ru_RU")` — нужна установленная в системе локаль.

Группировка по периодам: resample и Grouper

`resample` — это `groupby` для временного ряда с датой в индексе: делит данные на периоды (день, неделя, месяц) и агрегирует их. Пустые периоды появляются в результате с нулём или NaN.

```
ts = df.set_index('date')['revenue']
print(ts.resample('ME').sum())                                  # by months
print(ts.resample('W').sum().head(3))                           # by weeks
print(df.groupby(df['date'].dt.to_period('M'))['revenue'].sum())   # same via period
print(df.groupby(pd.Grouper(key='date', freq='ME'))['revenue'].mean().round(0))
print(df.groupby([pd.Grouper(key='date', freq='ME'), 'region'])['revenue'].sum()
        .unstack(fill_value=0))                                  # months x regions
```

```
# вывод
date
2026-01-31    3900
2026-02-28    2460
2026-03-31    1600
2026-04-30    5160
Freq: ME, Name: revenue, dtype: int64
date
2026-01-11    1900
2026-01-18    2000
2026-01-25       0
Freq: W-SUN, Name: revenue, dtype: int64
date
2026-01    3900
2026-02    2460
2026-03    1600
2026-04    5160
Freq: M, Name: revenue, dtype: int64
date
2026-01-31    1950.0
2026-02-28    1230.0
…
```

Алиас	Период	Алиас	Период
`D`	календарный день	`QE` / `QS`	конец / начало квартала
`B`	рабочий день	`YE` / `YS`	конец / начало года
`W`	неделя (конец — воскресенье)	`h`	час
`ME` / `MS`	конец / начало месяца	`min`	минута

> **Важно.** Алиасы `ME`, `QE`, `YE`, `h` используются в pandas 2.2+ (раньше `M`, `Q`, `A`, `H`). Если на вашей версии старый алиас выдаёт предупреждение или ошибку — замените его на новый.

Часовые пояса

```
t = pd.Timestamp('2026-03-15 12:00', tz='Europe/Moscow')      # aware timestamp
print(t, '->', t.tz_convert('UTC'))
s = Series(pd.to_datetime(['2026-03-15 12:00', '2026-03-15 18:30']))
print(s.dt.tz_localize('Europe/Moscow').dt.tz_convert('UTC'))   # naive -> aware -> UTC
```

`tz_localize` присваивает часовой пояс «наивным» датам (без зоны), `tz_convert` переводит дату с зоной в другую зону. Сравнивать и вычитать можно только даты одного вида — либо обе с зоной, либо обе без.

Строковые методы .str

Доступ к строковым методам даёт аксессор `.str`. Методы применяются ко всему столбцу сразу, а пропуски (NaN) обрабатываются без ошибок и остаются пропусками.

Очистка и преобразование текста

```
names = Series([' Anna Petrova ', 'oleg  ivanov', 'IVAN SIDOROV', None])
print(names.str.strip().str.title().tolist())                # trim + capitalize words
print(names.str.lower().str.len().tolist())                  # length of the text
print(names.str.replace(r'\s+', ' ', regex=True).str.strip().tolist())   # extra spaces
print(names.str.strip().str[:4].tolist())                    # first 4 characters
print(Series(['7', '42']).str.zfill(4).tolist())             # pad with zeros
```

Поиск и фильтрация по тексту

```
print(names.str.contains('ov', case=False, na=False).tolist())   # na=False for filtering
print(names.str.startswith('Anna', na=False).tolist())
print(names.str.strip().str.endswith('OV', na=False).tolist())
print(names[names.str.contains('IVAN', case=False, na=False)].tolist())  # use as a mask
```

Важно. Для обычного текста добавляйте `regex=False` (`.str.contains("a.b", regex=False)`): по умолчанию строка трактуется как регулярное выражение, и спецсимволы `.`, `+`, `(` работают не так, как ожидается.

Разбиение и регулярные выражения

```
fio = Series(['Petrova Anna', 'Ivanov Oleg'])
print(fio.str.split(' ', expand=True))                # parts -> separate columns
print(fio.str.split(' ').str[0].tolist())             # only the first part

codes = Series(['A-100', 'B-250', 'C-7'])
print(codes.str.extract(r'(?P<letter>[A-Z])-(?P<num>\d+)'))   # groups -> columns
print(codes.str.findall(r'\d').tolist())                       # all matches
print(Series(['a,b', 'b,c', 'a']).str.get_dummies(sep=','))    # tags -> 0/1 columns
```

```
# вывод
         0     1
0  Petrova  Anna
1   Ivanov  Oleg
['Petrova', 'Ivanov']
  letter  num
0      A  100
1      B  250
2      C    7
[['1', '0', '0'], ['2', '5', '0'], ['7']]
   a  b  c
0  1  1  0
1  0  1  1
2  1  0  0
```

Типичная задача: цена в виде текста

```
raw = Series(['1 250,50 руб', '980 руб', '2 000,00 руб'])
price = (raw.str.replace('руб', '', regex=False)
            .str.replace(r'\s', '', regex=True)        # all whitespace, incl. non-breaking
            .str.replace(',', '.', regex=False)
            .astype(float))
print(price.tolist(), price.sum())
```

```
# вывод
[1250.5, 980.0, 2000.0] 4230.5
```

Сдвиги и скользящие окна

Эти методы нужны для временных рядов и динамики: сравнить значение с предыдущим, посчитать рост, скользящее среднее, накопленный итог.

```
sales = Series([100, 120, 90, 150, 130, 170],
               index=pd.date_range('2026-01-31', periods=6, freq='ME'), name='sales')
res = DataFrame({
    'sales': sales,
    'prev': sales.shift(1),                          # value of the previous period
    'diff': sales.diff(),                            # change = sales - prev
    'pct': (sales.pct_change() * 100).round(1),      # growth, %
    'ma3': sales.rolling(3).mean().round(1),         # moving average, window 3
    'cum': sales.cumsum(),                           # running total
    'cummax': sales.cummax(),                        # running maximum
})
print(res)
```

```
# вывод
            sales   prev  diff   pct    ma3  cum  cummax
2026-01-31    100    NaN   NaN   NaN    NaN  100     100
2026-02-28    120  100.0  20.0  20.0    NaN  220     120
2026-03-31     90  120.0 -30.0 -25.0  103.3  310     120
2026-04-30    150   90.0  60.0  66.7  120.0  460     150
2026-05-31    130  150.0 -20.0 -13.3  123.3  590     150
2026-06-30    170  130.0  40.0  30.8  150.0  760     170
```

```
print(sales.rolling(3, min_periods=1).sum().tolist())     # do not wait for a full window
print(sales.expanding().mean().round(1).tolist())         # window grows from the start
print(sales.ewm(span=3).mean().round(1).tolist())         # exponential smoothing
print(sales.pct_change(periods=3).round(2).tolist())      # compare with 3 periods ago

# inside groups: shift / cumsum / rolling are applied to each group separately
df = base_df.copy()
df['prev_in_region'] = df.groupby('region')['revenue'].shift(1)
df['cum_in_region'] = df.groupby('region')['revenue'].cumsum()
cols = ['region', 'revenue', 'prev_in_region', 'cum_in_region']
print(df[cols].sort_values('region').head(4))
```

Категориальные данные

Тип `category` хранит повторяющиеся значения компактно (числовые коды и справочник категорий) и позволяет задать порядок значений. Подходит для столбцов вроде «регион», «статус», «размер».

```
size = Series(['low', 'high', 'mid', 'low']).astype(
    pd.CategoricalDtype(['low', 'mid', 'high'], ordered=True))   # ordered categories
print(size.sort_values().tolist())          # by the order, not alphabetically
print((size > 'low').tolist())              # comparison works for ordered categories
print(size.cat.codes.tolist(), size.cat.categories.tolist())
print(size.value_counts())

big = Series(['Moscow', 'Kazan', 'Perm'] * 30000)
print(big.astype('category').memory_usage(deep=True) < big.memory_usage(deep=True))
```

MultiIndex

Многоуровневый индекс появляется после группировки по нескольким ключам, после `pivot_table` или `set_index([...])`. Работать с ним проще, если вовремя вернуть уровни в столбцы.

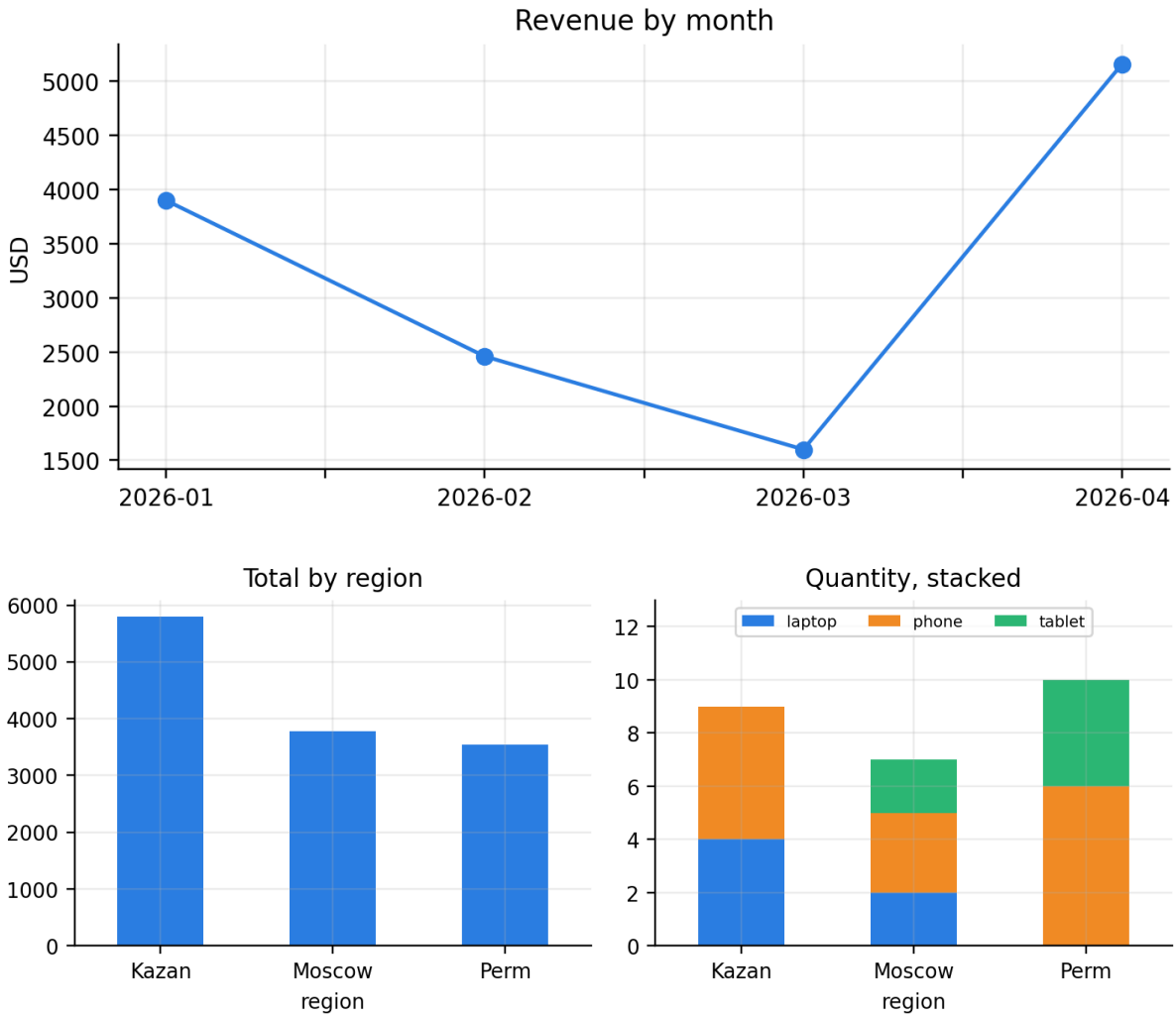
```
df = base_df.copy()
res = df.groupby(['region', 'product'])['qty'].sum()
print(res.index.names)
print(res.loc['Kazan'])                          # by the first level
print(res.loc[('Kazan', 'laptop')])              # by both levels
print(res.xs('phone', level='product'))          # cross-section on the second level
print(res.groupby(level='region').sum())         # aggregate by a level
print(res.reset_index().head(3))                 # levels -> columns (usually the best)
```

```
agg = df.groupby('region').agg({'qty': ['sum', 'mean'], 'revenue': ['min', 'max']})
print(agg.columns.tolist())                      # two-level columns
agg.columns = ['_'.join(c) for c in agg.columns]  # flatten: qty_sum, qty_mean, ...
print(agg)
```

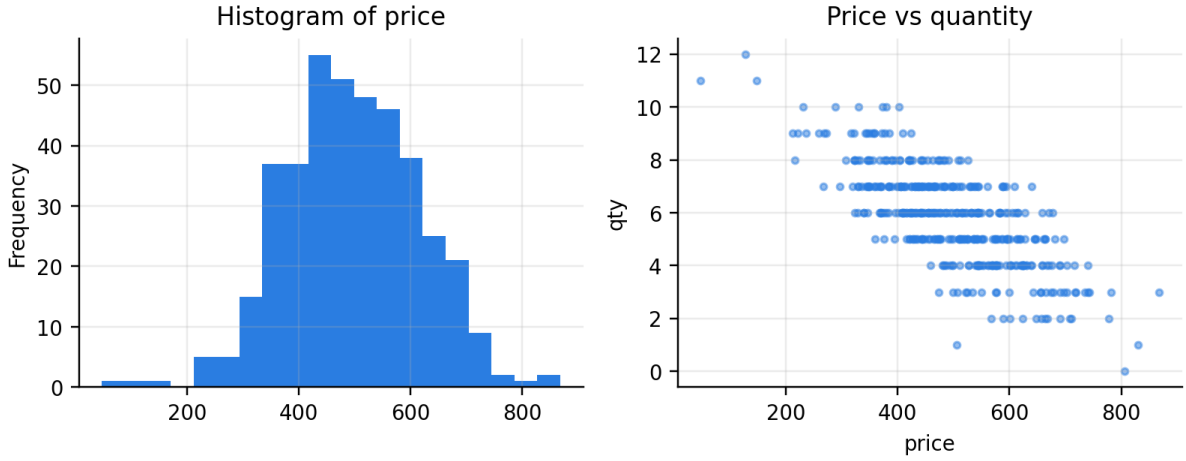
Быстрая визуализация

Метод `.plot()` строит графики на основе matplotlib прямо из Series и DataFrame: индекс идёт на ось X, столбцы становятся линиями или столбиками. Тип графика задаёт параметр `kind`.

Линия и столбцы



Распределение и связь двух величин



kind	График	Когда использовать
`line`	линия	динамика во времени
`bar`, `barh`	столбцы (вертикальные / горизонтальные)	сравнение категорий
`hist`	гистограмма	распределение числового признака
`box`	ящик с усами	разброс и выбросы по группам
`scatter`	точки	связь двух числовых признаков
`pie`	круговая	доли (до 5–6 сегментов)
`area`	область	накопление во времени

Особенности pandas 3.0

Примеры в этом конспекте проверены на **pandas 3.0**. Если у вас pandas 2.x, большая часть кода работает так же, но есть несколько заметных отличий, о которых важно знать при чтении чужого кода и ошибок.

Что	pandas 2.x	pandas 3.0
Тип строк	`object`	отдельный тип `str`
Копии и представления	иногда изменение «среза» меняет исходную таблицу	Copy-on-Write: результат любой операции — независимая копия
Цепочка присваиваний	`SettingWithCopyWarning`	не работает совсем; предупреждения нет, правку нужно делать через `loc`
Метод `applymap`	устарел с 2.1	заменён на `DataFrame.map`
Единицы времени дат	всегда наносекунды (`datetime64[ns]`)	определяются по данным (`us`, `s`, ...)

```
print(pd.__version__)
print(Series(['a', 'b']).dtype)            # 'str' in pandas 3, 'object' in pandas 2

a = DataFrame({'x': [1, 2, 3]})
b = a['x']
b.iloc[0] = 100                             # change the "copy"...
print(a['x'].tolist())                      # ...original is untouched (Copy-on-Write)
```

```
# вывод
3.0.2
str
[1, 2, 3]
```

Рекомендация. Старый код с `inplace=True` и цепочками вроде `df[df["a"] > 1]["b"] = 0` лучше переписать: присваивайте результат (`df = df.drop(...)`) и меняйте значения через `df.loc[условие, "b"] = 0`.

Частые ошибки и подводные камни

Присваивание по цепочке и and / or

```
df = base_df.copy()
df[df['qty'] > 3]['price'] = 0                # WRONG: changes a temporary copy
print(df['price'].tolist())                   # nothing changed

df.loc[df['qty'] > 3, 'price'] = 0            # RIGHT: one loc with condition and column
print(df['price'].tolist())

try:
    df[(df['qty'] > 2) and (df['price'] > 300)]    # WRONG: and/or do not work for series
except ValueError as e:
    print('ValueError:', str(e)[:48])
```

```
# вывод
[950, 400, 420, 300, 980, 310, 390, 940]
[950, 0, 420, 0, 980, 310, 0, 940]
ValueError: The truth value of a Series is ambiguous. Use a.
```

loc и iloc, inplace, NaN

```
df = base_df.copy()
print(df.loc[1:3].index.tolist(), df.iloc[1:3].index.tolist())    # [1,2,3] vs [1,2]

res = df.sort_values('qty', inplace=True)       # inplace methods return None
print(res)

print(np.nan == np.nan, np.isnan(np.nan))       # NaN is not equal to anything
print(0.1 + 0.2 == 0.3, np.isclose(0.1 + 0.2, 0.3))   # floats: compare with isclose

try:
    Series([1, np.nan, 3]).astype(int)          # int cannot hold NaN
except ValueError as e:
    print('ValueError')
print(Series([1, np.nan, 3]).astype('Int64').tolist())    # nullable integer can
```

Даты и повторяющиеся ключи

```
print(pd.to_datetime('03/04/2026'))                  # month first: 4th of March
print(pd.to_datetime('03/04/2026', dayfirst=True))     # day first: 3rd of April

k1 = DataFrame({'k': [1, 1], 'x': [1, 2]})
k2 = DataFrame({'k': [1, 1], 'y': [3, 4]})
print(len(k1), len(k2), '->', len(k1.merge(k2, on='k')))   # 2 x 2 = 4 rows!

print(Series(['a', None, 'a']).value_counts(dropna=False).to_dict())  # count NaN too
```

Скорость и память

```
n = Series(range(1000))
small = pd.to_numeric(n, downcast='integer')            # smallest suitable int type
print(n.dtype, '->', small.dtype)
print(round(n.memory_usage(index=False) / small.memory_usage(index=False), 1), 'x smaller')
```

- Читайте только нужные столбцы: `usecols=[...]`; для больших файлов — `chunksize` или формат parquet.
- Не используйте `iterrows` и `apply(axis=1)`, если задача решается векторно: разница в скорости десятки и сотни раз.
- Повторяющийся текст переводите в `category`, числа — в меньшие типы (`downcast`).
- Размер таблицы в памяти: `df.memory_usage(deep=True).sum()`.
- Не склеивайте таблицу в цикле через `pd.concat` на каждой итерации: соберите список и вызовите `concat` один раз.

Шпаргалка

Загрузка, осмотр, типы

Задача	Код
Прочитать CSV / Excel	`pd.read_csv("f.csv", sep=";", parse_dates=["d"])`, `pd.read_excel("f.xlsx")`
Форма, типы, память	`df.shape`, `df.dtypes`, `df.info()`
Первые / случайные строки	`df.head()`, `df.tail()`, `df.sample(5)`
Статистика	`df.describe()`, `df["a"].value_counts()`
Сменить тип	`astype("float")`, `pd.to_numeric(s, errors="coerce")`, `pd.to_datetime(s)`
Записать	`df.to_csv("f.csv", index=False)`, `df.to_excel("f.xlsx", index=False)`

Выборка и фильтрация

Задача	Код
Столбец / несколько	`df["a"]`, `df[["a", "b"]]`
По меткам / по позициям	`df.loc[строки, столбцы]`, `df.iloc[строки, столбцы]`
По условию	`df[(df["a"] > 1) & (df["b"] == "x")]`, `df.query("a > 1")`
Из списка / диапазон	`df["a"].isin([1, 2])`, `df["a"].between(1, 5)`
Текст содержит	`df["a"].str.contains("abc", case=False, na=False)`
Топ-N	`df.nlargest(3, "a")`, `df.sort_values("a").head(3)`
Убрать столбцы / строки	`df.drop(columns=["a"])`, `df.drop(index=[0, 1])`

Преобразование и пропуски

Задача	Код
Новый столбец	`df["c"] = df["a"] * df["b"]`, `df.assign(c=...)`
Условие	`np.where(усл, x, y)`, `np.select([усл1, усл2], [x1, x2], default=y)`
Словарь замен	`df["a"].map({"x": 1})`, `df["a"].replace({"x": 1})`
Интервалы	`pd.cut(s, bins=[...])`, `pd.qcut(s, q=4)`
Пропуски: найти	`df.isna().sum()`
Пропуски: удалить / заполнить	`df.dropna(subset=["a"])`, `df.fillna({"a": 0})`, `s.ffill()`
Дубликаты	`df.duplicated()`, `df.drop_duplicates(subset=["a"])`
Переименовать	`df.rename(columns={"a": "b"})`

Группировка, сводные, соединения

Задача	Код
Агрегация по группам	`df.groupby("k").agg(n=("a", "count"), s=("b", "sum"))`
Показатель в строке группы	`df.groupby("k")["a"].transform("sum")`
Сводная таблица	`df.pivot_table(index="a", columns="b", values="c", aggfunc="sum")`
Таблица частот	`pd.crosstab(df["a"], df["b"], normalize="index")`
Широкая → длинная	`df.melt(id_vars="k")`; обратно — `df.pivot(...)`
Склеить таблицы	`pd.concat([a, b], ignore_index=True)`
Соединить по ключу	`a.merge(b, on="k", how="left", validate="many_to_one")`
Индекс ↔ столбец	`df.set_index("k")`, `df.reset_index()`

Даты, текст, окна

Задача	Код
Компоненты даты	`df["d"].dt.year`, `.dt.month`, `.dt.day_name()`, `.dt.strftime("%d.%m.%Y")`
Сдвиг даты	`df["d"] + pd.Timedelta(days=7)`, `+ pd.DateOffset(months=1)`
Период (месяц)	`df["d"].dt.to_period("M")`, `s.resample("ME").sum()`
Текст: чистка	`.str.strip().str.lower().str.replace(...)`
Текст: разбить / вытащить	`.str.split(" ", expand=True)`, `.str.extract(r"(\d+)")`
Динамика	`s.shift(1)`, `s.diff()`, `s.pct_change()`
Окна	`s.rolling(3).mean()`, `s.expanding().max()`, `s.cumsum()`